
🧪 Kubernetes Resource Requests & Limits Lab

Objective:
Understand how Kubernetes manages CPU and memory for pods using **Requests** and **Limits**, and observe how they affect scheduling, runtime behavior, throttling, and pod crashes.

Prerequisites:

- Kubernetes cluster with kubectl access
- Metrics Server installed for kubectl top commands
- Basic understanding of pods and deployments

Part 1: Key Concepts

Kubernetes allows you to define **resource requirements** for pods. These help the scheduler place pods efficiently and prevent a pod from consuming excessive resources.

1.1 Definitions

Concept	Description	Notes
Requests	Minimum guaranteed amount of CPU or memory a container needs	Scheduler ensures node has enough resources; CPU can burst beyond requests if available; memory cannot be guaranteed if requests are too low
Limits	Maximum allowed CPU or memory a container can use	Kubernetes enforces this limit; exceeding CPU → throttling; exceeding memory → container killed (OOMKilled)

Notes:

- **CPU**: compressible; throttled when over limit.
- **Memory**: non-compressible; exceeding limit kills the pod.
- Pods **without requests or limits** can be scheduled anywhere and may destabilize nodes.
- Pods **with only limits** may fail if node does not have enough free resources.
- Pods **with only requests** can schedule safely but may overconsume CPU.
- Recommended: **set both requests and limits** for production workloads.

1.2 Example Scenario

Imagine deploying a Node.js API server:

- Usually requires 256Mi memory and 0.25 CPU
- Under load, it may spike to 512Mi memory and 0.5 CPU

You would define:

```
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

Notes:

- Guarantees minimum resources (requests) → safe scheduling
- Caps maximum resources (limits) → prevents node from being overwhelmed

---

Part 2: Hands-On Lab Pods

We will create **four types of pods** to see behavior under different resource configurations.

2.1 Pod with No Requests or Limits

File: no-resources-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: no-resources-pod
spec:
  containers:
  - name: nginx
    image: nginx
```

Apply:

```
kubectl apply -f no-resources-pod.yaml
kubectl get pods
```

Expected Behavior:

- Pod schedules on any available node.
- CPU/memory usage is uncontrolled → may affect other pods.
- **Notes:** Avoid this in production; this can lead to resource starvation.

2.2 Pod with Requests Only

File: requests-only-pod.yaml

```
apiVersion: v1
```

```
kind: Pod
metadata:
  name: requests-only-pod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "128Mi"
        cpu: "250m"
```

Apply:

```
kubectl apply -f requests-only-pod.yaml
kubectl get pods
```

Expected Behavior:

- Scheduler ensures node has at least **128Mi memory** and **0.25 CPU** free.
- Pod may **burst CPU** if node has extra free capacity.
- Memory usage can grow if node allows, but node may not guarantee more than requested.
- **Notes:** Requests help the scheduler place pods safely, but overuse is possible if no limits are set.

2.3 Pod with Limits Only

File: limits-only-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: limits-only-pod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Apply:

```
kubectl apply -f limits-only-pod.yaml
kubectl get pods
```

Expected Behavior:

- Pod may be scheduled on a node even if CPU/memory is tight.
- CPU is throttled if usage exceeds 500m → slower performance but still running.
- Memory over 128Mi → pod killed (OOMKilled).
- **Notes:** Limits protect nodes from resource abuse but do not guarantee scheduling if node is busy.

2.4 Pod with Requests and Limits

File: cpu-throttle-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: cpu-throttle-pod
spec:
  containers:
```

```
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "128Mi"
        cpu: "500m"
      limits:
        memory: "256Mi"
        cpu: "500m"
```

Apply:

kubectl apply -f cpu-throttle-pod.yaml
kubectl get pods

Expected Behavior:

- Pod is guaranteed **128Mi memory** and **0.5 CPU**.
- CPU can burst to **500m** (in this example, request = limit).
- Memory cannot exceed **256Mi**; if it does → pod killed.
- **Notes:** Best practice for production workloads → safe scheduling + prevents overuse.

Part 3: Observing Resource Usage

3.1 Check Metrics Server

kubectl rollout restart deployment metrics-server -n kube-system
kubectl get pods -n kube-system | grep metrics

Notes: Metrics server must be running to use kubectl top.

3.2 Observe Pod Metrics

kubectl top pod no-resources-pod
kubectl top pod requests-only-pod
kubectl top pod cpu-throttle-pod

Notes:

- Requests determine scheduling guarantees.
- Limits determine maximum usage (CPU throttling, OOMKilled for memory).

3.3 Observe Node Metrics

kubectl top node

Notes: Helps identify stressed nodes, over-allocated nodes, or pods consuming unexpected resources.

3.4 CPU Throttling Experiment

1. Exec into cpu-throttle-pod:

kubectl exec -it cpu-throttle-pod -- sh

2. Generate CPU load:

yes > /dev/null &

3. Observe usage:

kubectl top pod cpu-throttle-pod -w

Expected Observation:

- CPU usage capped at **500m** → throttled.
- Memory stays below limit unless stress applied.

Notes: CPU throttling slows down the container but keeps it alive. Memory overuse kills the pod.

Part 4: Summary Table

Pod Type	Requests	Limits	Expected Behavior	Notes
No Requests/Limits	❌	❌	Can schedule anywhere; may overconsume	Unsafe for production
Requests Only	✅	❌	Guaranteed resources; CPU can burst	Helps scheduler; memory may overuse
Limits Only	❌	✅	Maximum capped; may fail if node busy	Protects nodes; no guaranteed scheduling
Requests + Limits	✅	✅	Safe scheduling + capped usage	Recommended for production

---

Part 5: Cleanup

kubectl delete pod no-resources-pod requests-only-pod limits-only-pod cpu-throttle-pod

---

🧪 Kubernetes Resource Requests & Limits Lab (Updated with Metrics Server)

Part 0: What is Metrics Server?

Metrics Server is a cluster-wide aggregator of resource usage data.
It collects metrics like CPU and memory usage from kubelets on each node and exposes them through the Metrics API.

Why it's important:

- kubectl top pod and kubectl top node rely on Metrics Server.
- Enables Horizontal Pod Autoscalers (HPA) and Vertical Pod Autoscalers (VPA) to scale pods based on CPU/memory usage.
- Lightweight alternative to Prometheus for real-time metrics.

Part 0.1: Installing Metrics Server

1. Check if Metrics Server is already installed

```
kubectl get deployment metrics-server -n kube-system
```

- If you see a deployment → already installed.
- If not → proceed with installation.

2. Install Metrics Server using official YAML

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

3. Verify Metrics Server pods are running

```
kubectl get pods -n kube-system | grep metrics-server
```

- STATUS should be Running for at least one pod.

4. Check the Metrics API

```
kubectl get apiservices | grep metrics
```

- v1beta1.metrics.k8s.io should show True in the AVAILABLE column.

5. Restart Metrics Server if needed

```
kubectl rollout restart deployment metrics-server -n kube-system
```

6. Test metrics collection

```
kubectl top node
```

`kubectl top pod`

- **Should now show CPU and memory usage for nodes and pods.**

Notes:

- **Metrics Server does not store historical metrics. For long-term metrics, use Prometheus.**

- **Ensure kubelets are configured with `--read-only-port=10255` disabled (default for newer versions). Metrics Server talks to kubelet via HTTPS.**

Best Practices

- Always set **both requests and limits** for production pods.
- Monitor with **Metrics Server, Prometheus, Grafana**.
- Tune requests and limits based on **real workload observations**.
- Use **ResourceQuotas** and **LimitRanges** at namespace level to prevent cluster abuse.
- Combine with **Horizontal/Vertical Pod Autoscalers** for dynamic scaling.

✅ **Conclusion:**
Requests and limits help Kubernetes balance **efficient scheduling**, **resource guarantees**, and **node stability**. This lab demonstrates the practical effects of different configurations on pod behavior.
